

Module 12 — Benchmark Harness

notebooks/recommendation_evaluation.py • 462 lines

In one sentence

Compares every model fairly. This is where the two biggest bugs were found.

1. What this module does

This module answers the question the whole project exists to answer: which model is best? It runs all six on the same customers, the same split and the same metrics.

Two rules make the comparison trustworthy. First, the last 90 days are held out and no model sees them. Second - and this is the part people usually skip - **every model is retrained from scratch on the training period only**. Reusing the saved artifacts would be much faster, but those were fitted on the full history including the test window, so every score would be inflated.

Running this is what exposed the broken NCF and the underperforming hybrid.

2. Concepts covered

Every idea this module uses, in plain terms.

Concept	In simple terms	Where it is used here
Train / test split	Hold data back to test honestly	Last 90 days held out
Time-based split	Test on the future, not random rows	<code>time_based_split()</code>
Data leakage	Model accidentally sees the answers	The thing this module prevents
Assertions	Crash rather than continue if wrong	The leakage checks
Ground truth	What actually happened	<code>build_ground_truth()</code>
Relevance labelling	Deciding what counts as a hit	Rated 4+ OR purchased
Candidate exclusion	Do not recommend what they already bought	The seen set
Cohort sampling	Test on a representative subset	800 customers
Fair comparison	Same data, same metrics, same users	All models share one interface

3. How it works, step by step

1. **Split by time** at 90 days before the end, and **assert** that no test row is at or before the cutoff.
2. **Build ground truth** - for each customer, which products they rated 4+ or purchased during the test period.
3. **Rebuild every model on the training period only** - SVD, item similarity, popularity ranking, the popularity lookup.
4. **Sample 800 customers** who have at least one relevant test item.
5. **Generate recommendations from each of the six models**, excluding everything the customer touched during training.
6. **Compute all metrics** at K = 5, 10 and 20.

7. **Compute the beyond-accuracy metrics** - coverage, long-tail share, novelty, diversity.
8. **Save results** to CSV and print the headline.

4. Key functions

Function	What it does
<code>build_ground_truth()</code>	What each customer actually found relevant
<code>build_train_artifacts()</code>	Retrains every model on train data only
<code>recommend_popularity()</code>	Baseline scorer
<code>recommend_item_cf()</code>	Item-based CF scorer
<code>recommend_svd_model()</code>	SVD scorer
<code>recommend_content()</code>	Content scorer
<code>recommend_ncf_model()</code>	Deep learning scorer
<code>recommend_hybrid_eval()</code>	Hybrid scorer, rebuilt leak-free

5. Inputs, outputs and how to run

	Detail
Inputs	All raw data plus the trained NCF
Outputs	<code>recommendation_evaluation_results.csv</code> , <code>beyond_accuracy_results.csv</code>
Split	Train 96,656 events • Test 41,689 events
Runtime	About 6 minutes
Run with	<code>python notebooks/recommendation_evaluation.py</code>

Final results at K = 10

Model	Precision	Recall	NDCG	Coverage
Hybrid	0.0628	0.1372	0.1095	20.2%
Item-based CF	0.0579	0.1241	0.1044	22.2%
NCF	0.0375	0.0858	0.0663	1.3%
Popularity	0.0375	0.0843	0.0645	1.3%
SVD	0.0278	0.0511	0.0462	9.6%
Content	0.0073	0.0161	0.0116	75.4%

6. Key code

notebooks/recommendation_evaluation.py — build_train_artifacts()

```

92 | def build_train_artifacts(train_df, items_df):
93 |     """Rebuild every model from the training period alone."""
94 |     print(" rebuilding models on the training period only ...")
95 |
96 |     user_item_matrix = create_user_item_matrix(train_df)
97 |     normalized_matrix = create_normalized_user_item_matrix(train_df)
98 |     implicit_matrix = create_implicit_feedback_matrix(train_df)
99 |     item_popularity = create_item_popularity_features(train_df, items_df)
100 |
101 |     popularity_lookup = build_popularity_lookup(item_popularity)
102 |
103 |     svd, user_factors, item_factors = create_svd_model(normalized_matrix)
104 |     predicted_ratings = create_predicted_rating_matrix(
105 |         normalized_matrix, user_factors, item_factors
106 |     )
107 |
108 |     item_similarity = pd.DataFrame(
109 |         cosine_similarity(user_item_matrix.T).astype(np.float32),
110 |         index=user_item_matrix.columns,
111 |         columns=user_item_matrix.columns,
112 |     )
113 |
114 |     popularity_ranking = (
115 |         item_popularity.sort_values("interaction_count", ascending=False)["item_id"].tolist()
116 |     )
117 |     ...

```

This is the function that actually enforces the no-leakage rule.

7. Things to remember

Bug found here #1: the deep model could not rank at all

The first run gave the NCF **NDCG@10 = 0.0000** with 0.6% catalogue coverage - it returned the same 14 products to every customer. The cause was that the explicit rating model was being used for ranking. Switching to the implicit model trained with negative sampling took it to 0.0576.

Bug found here #2: the hybrid lost to its own input

The second run gave the hybrid 0.0760 against item-based CF's 0.1044. The hybrid was **worse than one of the models inside it**. The collaborative slot held SVD, the weakest signal, while item-based CF was not in the fusion at all. Re-weighting fixed it and took the hybrid to the top.

Why leakage is checked with assertions rather than trusted

If a future change ever breaks the temporal separation, the benchmark crashes instead of quietly reporting inflated numbers. A silent leak is far more dangerous than a crash, because the results still look plausible.

8. Likely questions

Q. What is data leakage and how do you prevent it?

A. It is when the model gets to see information it would not have at prediction time. Here that means test-period behaviour. I prevent it three ways: a strict time-based split, retraining every model on training data only rather than reusing saved artifacts, and assertions in the code that crash if the separation is ever broken.

Q. Why retrain everything instead of reusing the saved models?

A. Because the saved models were fitted on the full history, including the test window. Scoring them against that same window would be marking their own homework. Retraining takes longer but it is the only honest option.

Q. Why only 800 customers?

A. Scoring the full catalogue for all 5,800 customers across six models is slow and adds nothing - the metric standard errors are already small at 800. Customers with no relevant test items are excluded too, since including them drags every score toward zero by an amount that depends on the split rather than the model.